



Introducción a React.js

Guía Didáctica para Desarrollo de Software

Contenidos de esta Guía

01. ¿Qué es React.js?
02. Cómo funciona React — Componentes y Virtual DOM
03. JSX: HTML dentro de JavaScript
04. MPA vs SPA — ¿Por qué React?
05. Componentes: Los bloques de React
06. Hooks — Introducción
07. useState — Manejando datos que cambian
08. useEffect — Efectos secundarios
09. useContext — Estado global sin complicaciones
10. Librerías complementarias: React Hook Form y Axios

01

¿Qué es React.js?

La biblioteca más popular para construir interfaces web

React es una **biblioteca de JavaScript de código abierto**, creada por **Meta (Facebook) en 2013**, orientada a construir **interfaces de usuario interactivas**. Su gran fortaleza es dividir la UI en piezas pequeñas y reutilizables llamadas **componentes**.

¿Por qué React domina el mercado?

- Utilizado por Meta, Netflix, Airbnb, Uber y miles de empresas
- El skill de frontend más demandado en ofertas laborales
- Ecosistema enorme: React Native (móvil), Next.js (web con SSR), Remix
- Comunidad activa y documentación excelente en react.dev
- Curva de aprendizaje moderada, alta recompensa profesional

Sus características principales

- **Declarativo:** describes QUÉ mostrar, React decide CÓMO actualizarlo
- **Basado en componentes:** bloques reutilizables con su propia lógica y apariencia
- **JSX:** escribe HTML dentro de JavaScript de forma natural
- **Virtual DOM:** actualiza solo lo que cambió, no toda la pantalla
- **SPA:** la app carga una sola vez y navega sin recargas de página
- **Hooks:** funciones especiales para gestionar estado y ciclo de vida

React: Imperativo vs. Declarativo

Este es el cambio mental más importante al aprender React:

JavaScript puro (Imperativo)	React (Declarativo)
Le decís al navegador CÓMO cambiar cada elemento	Describís QUÉ querés mostrar según el estado actual
Usás getElementById, addEventListener, innerHTML...	React decide por sí solo qué partes actualizar
A medida que la app crece, el código se vuelve difícil de mantener	El código escala mejor: separás UI de lógica
Tenés que pensar en cada paso para modificar la pantalla	Solo te preocupás por los datos, React maneja el DOM

Ejemplo comparativo: un contador

Ambos hacen lo mismo, pero observá la diferencia de enfoque:

```
JavaScript puro — hay que indicar cada paso
// — JAVASCRIPT PURO (imperativo) —
let count = 0;
const btn = document.getElementById('btn');
const display = document.getElementById('display');

btn.addEventListener('click', () => {
  count++;
  display.innerText = count; // manipulamos el DOM a mano
});
```

```
React — solo describís el estado y la UI resultante
// — REACT (declarativo) —
import { useState } from 'react';

function Contador() {
  const [count, setCount] = useState(0); // estado

  return (
    <div>
      <p>Clicks: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Incrementar
      </button>
    </div>
  );
}

// React actualiza el DOM automáticamente cuando 'count' cambia
```

💡 TIP

En React nunca tocarás directamente el DOM con `getElementById`. Solo modificás el estado y React hace el resto.

02

Cómo funciona React

Componentes y Virtual DOM

¿Qué es el Virtual DOM?

Cuando un estado cambia, React no actualiza toda la página. En cambio, trabaja con una **copia ligera del DOM en memoria** (el Virtual DOM), compara qué cambió respecto a la versión anterior y **aplica solo los cambios mínimos** al DOM real del navegador.

Flujo de actualización en React

1. El estado cambia → 2. React genera un nuevo Virtual DOM → 3. Compara con el Virtual DOM anterior (diffing) → 4. Aplica SOLO los cambios necesarios al DOM real → 5. La pantalla se actualiza eficientemente

DOM Real vs. Virtual DOM

DOM Real (JavaScript puro)	Virtual DOM (React)
Cada cambio redibuja partes de la pantalla Cálculos costosos de estilos y layout Puede producir parpadeos o lentitud El desarrollador controla cuándo y cómo actualizar	Los cambios se calculan en memoria, son rápidos Solo el mínimo indispensable llega al DOM real Interfaz fluida y sin parpadeos React se encarga de la optimización automáticamente

03

JSX: HTML dentro de JavaScript

La sintaxis que hace React tan intuitivo

JSX (JavaScript XML) es la extensión de sintaxis que permite escribir etiquetas similares a HTML dentro del código JavaScript. Los navegadores no entienden JSX directamente — herramientas como **Babel** o **Vite** lo transforman automáticamente en llamadas a `React.createElement()`.

```
JSX se transforma automáticamente — no tenés que preocuparte por esto
// Lo que vos escribís (JSX):
const elemento = <h1 className='titulo'>Hola Mundo</h1>;

// Lo que Babel genera internamente:
const elemento = React.createElement('h1', { className: 'titulo' }, 'Hola
Mundo');

// No necesitás escribir la segunda forma — JSX lo hace por vos
```

Reglas esenciales de JSX

1 · Un solo elemento raíz

```
// ✗ ERROR — dos elementos en el nivel raíz
return (
  <h1>Título</h1>
  <p>Párrafo</p>
);

// ✔ CORRECTO — envolver con un <div> o un fragmento <>
return (
  <>
    <h1>Título</h1>
    <p>Párrafo</p>
  </>
);
```

2 · className en lugar de class

```
// ✗ class es una palabra reservada en JavaScript
<div class='mi-caja'>...</div>

// ✔ Usar siempre className
<div className='mi-caja'>...</div>
```

3 · Expresiones JavaScript con llaves { }

```
function Saludo() {
  const nombre = 'Ana';
  const edad = 20;

  return (
```

```

    <div>
      <p>Hola, {nombre}!</p>           {/* variable */}
      <p>Tenés {edad} años</p>         {/* variable */}
      <p>El doble es {edad * 2}</p>     {/* expresión */}
      <p>{edad >= 18 ? 'Mayor' : 'Menor'} de edad</p>  {/* ternario */}
    </div>
  );
}

```

4 · Etiquetas siempre cerradas

```

// ❌ ERROR - etiquetas sin cerrar
<input type='text'>
<img src='foto.jpg'>

// ✅ CORRECTO - auto-cierre obligatorio
<input type='text' />
<img src='foto.jpg' />

```

💡 TIP

JSX no es HTML. Es JavaScript con azúcar sintáctico. Cada vez que tenés dudas, pensá: ¿esto sería válido en JavaScript?

04

MPA vs. SPA

¿Por qué las apps modernas usan React?

MPA — Multi Page Application	SPA — Single Page Application
Modelo tradicional (PHP, WordPress, etc.) Cada clic pide una página nueva al servidor El navegador recarga todo con cada navegación Mayor carga en el servidor Sensación de 'salto' entre páginas Bueno para: blogs, sitios de contenido estático	Modelo moderno (React, Vue, Angular...) La app carga UNA VEZ y actualiza partes del DOM La navegación es instantánea, sin recargas El servidor solo entrega datos (via API/JSON) Experiencia fluida como una app nativa Bueno para: dashboards, apps interactivas, redes sociales

React es ideal para SPAs: maneja la navegación con **React Router**, obtiene datos del servidor mediante llamadas **fetch / axios** y actualiza la interfaz sin recargar la página.

05

Componentes

Los bloques de construcción de toda app React

Un componente es una **función JavaScript que retorna JSX**. Pensalo como una pieza de LEGO: se puede reutilizar, combinar con otras, y cada una hace una tarea específica. El nombre siempre empieza en **mayúscula**.

Tu primer componente

Componente funcional básico

// Archivo: src/components/Saludo.jsx

```
function Saludo() {
  return (
    <div>
      <h2>¡Hola, bienvenido!</h2>
      <p>Esta es tu primera app en React.</p>
    </div>
  );
}
```

```
export default Saludo; // exportar para usarlo en otro archivo
```

Importar y reutilizar componentes

// Archivo: src/App.jsx — donde usamos el componente

```
import Saludo from './components/Saludo';
```

```
function App() {
  return (
    <div>
      <Saludo />      {/* se usa como etiqueta HTML */}
      <Saludo />      {/* se puede reutilizar N veces */}
    </div>
  );
}
```

```
export default App;
```

Props — Pasar datos de padre a hijo

Las **props** (propiedades) son argumentos que se pasan al componente como atributos HTML. El componente hijo las recibe y las usa, pero **nunca las modifica** (son de solo lectura).

Props — el mismo componente, datos diferentes

// Componente que recibe props

```
function TarjetaUsuario({ nombre, email }) {
  return (
    <div className='tarjeta'>
      <h3>{nombre}</h3>
      <p>{email}</p>
    </div>
  );
}
```

```

    </div>
  );
}

// Usarlo pasando diferentes datos cada vez
function App() {
  return (
    <div>
      <TarjetaUsuario nombre='Ana García' email='ana@email.com' />
      <TarjetaUsuario nombre='Luis Pérez' email='luis@email.com' />
    </div>
  );
}

// Resultado: dos tarjetas distintas con el mismo componente

```

Composición de componentes

Los componentes se pueden anidar para construir interfaces complejas a partir de piezas simples:

```

Composición — construyendo UIs complejas con piezas simples
// Árbol de componentes de una página típica

function App() {
  return (
    <div>
      <NavBar />           {/* componente de navegación */}
      <main>
        <Sidebar />       {/* menú lateral */}
        <ContenidoPrincipal>
          <ListaProductos />  {/* lista de items */}
          <Paginador />      {/* controles de página */}
        </ContenidoPrincipal>
      </main>
      <Footer />
    </div>
  );
}

```

☑️ RECUERDA

Regla de oro: cada componente debe tener UNA responsabilidad. Si hace demasiadas cosas, divídelo en varios.

06

Hooks

La revolución de los componentes funcionales

Los **Hooks** son funciones especiales de React (siempre empiezan con **use**) que permiten agregar funcionalidades como **estado, efectos y contexto** a los componentes funcionales. Fueron introducidos en **React 16.8 (2019)** y reemplazaron los complejos componentes de clase.

¿Qué problemas resuelven?

- **Antes de los Hooks:** había que usar clases con `this`, métodos de ciclo de vida complicados y herencia — código difícil de leer y mantener
- **Con Hooks:** todo es código funcional, más simple, más legible y reutilizable

Tabla de Hooks más importantes

Hook	Para qué sirve	Ejemplo de uso
<code>useState</code>	Guardar y actualizar datos locales	<code>const [n, setN] = useState(0)</code>
<code>useEffect</code>	Ejecutar código al montar o actualizar	<code>useEffect(() => { ... }, [dep])</code>
<code>useContext</code>	Leer datos globales sin prop drilling	<code>const val = useContext(Ctx)</code>
<code>useRef</code>	Referenciar el DOM sin re-renderizar	<code>const ref = useRef(null)</code>
<code>useMemo</code>	Memorizar cálculos costosos	<code>useMemo(() => calc(), [deps])</code>
<code>useCallback</code>	Memorizar funciones entre renders	<code>useCallback(() => fn(), [deps])</code>
<code>useReducer</code>	Estado complejo con acciones	<code>useReducer(reducer, initState)</code>

Reglas de los Hooks

⚠️ ATENCIÓN

Solo se pueden llamar Hooks al nivel raíz de un componente funcional. Nunca dentro de un `if`, `for`, o función anidada.

```
// ❌ INCORRECTO — Hook dentro de una condición
function Componente({ condicion }) {
  if (condicion) {
    const [valor, setValor] = useState(0); // ¡ERROR!
  }
}

// ✅ CORRECTO — Hook siempre al inicio del componente
```

```
function Componente({ condicion }) {  
  const [valor, setValor] = useState(0); // fuera de cualquier bloque  
  if (condicion) { /* ... */ }  
}
```

07

useState

Manejando datos que cambian en el tiempo

`useState` es el Hook fundamental de React. Permite que un componente **recuerde un valor entre renders**. Cuando ese valor cambia, React **re-renderiza el componente automáticamente** mostrando el nuevo dato en pantalla.

Sintaxis

Estructura de useState

```
import { useState } from 'react';

//          valor actual      función para cambiarlo      valor inicial
//          ↓                  ↓                            ↓
const [nombreEstado, setNombreEstado] = useState(valorInicial);
```

Ejemplo 1 — Contador simple

useState con un número

```
import { useState } from 'react';

function Contador() {
  const [cuenta, setCuenta] = useState(0); // inicia en 0

  return (
    <div>
      <p>Clicks: {cuenta}</p>
      <button onClick={() => setCuenta(cuenta + 1)}>+1</button>
      <button onClick={() => setCuenta(cuenta - 1)}>-1</button>
      <button onClick={() => setCuenta(0)}>Reiniciar</button>
    </div>
  );
}
```

Ejemplo 2 — Mostrar / ocultar

useState con un booleano - renderizado condicional

```
import { useState } from 'react';

function Toggle() {
  const [visible, setVisible] = useState(true); // booleano

  return (
    <div>
      <button onClick={() => setVisible(!visible)}>
        {visible ? 'Ocultar' : 'Mostrar'} mensaje
      </button>

      {visible && <p>¡Hola! Estoy visible 🙌</p>}
    </div>
  );
}
```

```

    </div>
  );
}

```

Ejemplo 3 — Campo de texto controlado

```

useState con un string - campo controlado
import { useState } from 'react';

function FormNombre() {
  const [nombre, setNombre] = useState(''); // string vacío

  return (
    <div>
      <input
        type='text'
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
        placeholder='Ingresá tu nombre'
      />
      {nombre && <p>Hola, {nombre}</p>}
    </div>
  );
}

```

Regla CRÍTICA al actualizar el estado

⚠️ ATENCIÓN

Nunca mutés el estado directamente. Siempre creá un nuevo valor con la función set.

```

Cómo actualizar arrays y objetos en el estado
// _____
// ARRAYS - siempre crear un nuevo array
// _____

// ❌ INCORRECTO - push muta el array original
items.push(nuevoItem);
setItems(items);

// ✅ CORRECTO - spread crea un array nuevo
setItems([...items, nuevoItem]);

// _____
// OBJETOS - siempre crear un nuevo objeto
// _____

// ❌ INCORRECTO - mutación directa
usuario.nombre = 'Juan';
setUsuario(usuario);

// ✅ CORRECTO - spread crea un objeto nuevo
setUsuario({ ...usuario, nombre: 'Juan' });

```


08

useEffect

Ciclo de vida y efectos secundarios

`useEffect` permite ejecutar código en momentos específicos del ciclo de vida: **al montar** el componente, **al actualizar** ciertos datos, o **al desmontar**. Se usa para llamadas a APIs, timers, suscripciones, etc.

Las tres formas de useEffect

Formas de uso de useEffect

```
import { useEffect } from 'react';

// ① Sin array → se ejecuta DESPUÉS de cada render
useEffect(() => {
  console.log('El componente se renderizó');
});

// ② Array vacío → se ejecuta UNA SOLA VEZ al montar
useEffect(() => {
  console.log('Componente montado — ideal para cargar datos');
}, []);

// ③ Con dependencias → se ejecuta cuando 'busqueda' cambia
useEffect(() => {
  console.log('La búsqueda cambió:', busqueda);
}, [busqueda]);
```

Ejemplo práctico — Cargar datos de una API

Patrón loading / error / data — el más común en React

```
import { useState, useEffect } from 'react';

function ListaUsuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(res => res.json())
      .then(data => {
        setUsuarios(data);
        setCargando(false);
      })
      .catch(err => {
        setError('No se pudieron cargar los usuarios');
        setCargando(false);
      });
  }, []); // [] = solo al montar el componente

  if (cargando) return <p>Cargando...</p>;
  if (error) return <p style={{ color: 'red' }}>{error}</p>;
```

```
return (
  <ul>
    {usuarios.map(u => (
      <li key={u.id}>{u.name} — {u.email}</li>
    ))}
  </ul>
);
}
```

Función de limpieza (cleanup)

`useEffect` puede retornar una función que se ejecuta al desmontar el componente. Se usa para cancelar timers o suscripciones.

Cleanup — evitar memory leaks

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log('tick');
  }, 1000);

  // Cleanup: se ejecuta cuando el componente se desmonta
  return () => clearInterval(timer);
}, []);
```

💡 TIP

Patrón obligatorio: siempre manejá los tres estados al hacer fetch — cargando, error y datos. Es la base de cualquier app real.

09

useContext

Estado global sin prop drilling

Cuando muchos componentes necesitan acceder al mismo dato (usuario logueado, tema, idioma), pasarlo como prop a través de múltiples niveles se llama **"prop drilling"** — es tedioso y propenso a errores. **useContext** resuelve esto creando un **canal de datos global** accesible desde cualquier componente sin pasar props.

Comparación: prop drilling vs. contexto

Prop drilling (problema)	useContext (solución)
App pasa usuario → Layout	App crea el contexto con el usuario
Layout pasa usuario → Sidebar	NavItem accede DIRECTAMENTE al contexto
Sidebar pasa usuario → NavItem	Los niveles intermedios no necesitan saber nada
NavItem finalmente usa usuario	Cambiar algo en un solo lugar actualiza todo
Cambiar algo requiere modificar TODOS los niveles	Ideal para: tema, idioma, usuario, permisos

Implementación paso a paso

```

Crear contexto y provider
// — PASO 1: Crear el contexto —————
// Archivo: src/context/TemaContext.jsx

import { createContext, useContext, useState } from 'react';

const TemaContext = createContext(null);

// — PASO 2: Crear el Provider (envuelve la app) —————
export function TemaProvider({ children }) {
  const [tema, setTema] = useState('claro');

  const cambiarTema = () => {
    setTema(t => t === 'claro' ? 'oscuro' : 'claro');
  };

  return (
    <TemaContext.Provider value={{ tema, cambiarTema }}>
      {children}
    </TemaContext.Provider>
  );
}

// — PASO 3: Custom Hook para consumirlo fácilmente —————
export function useTema() {
  return useContext(TemaContext);
}

```

```
}
```

```
Usar el contexto desde cualquier componente
// — PASO 4: Envolver la app con el Provider —
// Archivo: main.jsx
import { TemaProvider } from '../context/TemaContext';

ReactDOM.createRoot(document.getElementById('root')).render(
  <TemaProvider>
    <App />
  </TemaProvider>
);

// — PASO 5: Usar el contexto en cualquier componente —
// Archivo: src/components/BotonTema.jsx
import { useTema } from '../context/TemaContext';

function BotonTema() {
  const { tema, cambiarTema } = useTema();

  return (
    <button onClick={cambiarTema}>
      Tema actual: {tema} — Cambiar
    </button>
  );
}

// No importa cuán profundo esté en el árbol de componentes
```

10

Librerías complementarias

React Hook Form y Axios

React Hook Form — Formularios sin complicaciones

React Hook Form es una librería que simplifica enormemente la gestión y validación de formularios. En lugar de manejar cada campo con `useState`, deja que la librería se encargue del estado internamente, minimizando re-renders y reduciendo código.

1

Instalá la librería

2

Registrá los campos

3

Definí las validaciones

4

Manejá el submit

```
react-hook-form — menos código, validación integrada
// npm install react-hook-form

import { useForm } from 'react-hook-form';

function FormRegistro() {
  const {
    register,
    handleSubmit,
    formState: { errors }
  } = useForm();

  const onSubmit = (data) => {
    console.log('Datos:', data); // { nombre: '...', email: '...' }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>

      {/* Campo con validación integrada */}
      <input
        {...register('nombre', { required: 'El nombre es obligatorio' })}
        placeholder='Tu nombre'
      />
      {errors.nombre && <p>{errors.nombre.message}</p>}

      <input
        {...register('email', {
          required: 'El email es obligatorio',
          pattern: { value: /\S+@\S+/, message: 'Email inválido' }
        })}
        placeholder='Tu email'
      />
      {errors.email && <p>{errors.email.message}</p>}

      <button type='submit'>Registrarse</button>
    </form>
  );
}
```

Axios — Cliente HTTP para consumir APIs

Axios es la librería más usada para hacer peticiones HTTP en React. Simplifica el uso de `fetch`: maneja automáticamente JSON, errores HTTP y permite configuraciones globales.

Axios — GET y POST simplificados

```
// npm install axios

import axios from 'axios';
import { useState, useEffect } from 'react';

function Productos() {
  const [productos, setProductos] = useState([]);

  useEffect(() => {
    // GET — obtener datos
    axios.get('https://api.ejemplo.com/productos')
      .then(res => setProductos(res.data))
      .catch(err => console.error('Error:', err));
  }, []);

  const crearProducto = async () => {
    // POST — enviar datos
    const res = await axios.post('https://api.ejemplo.com/productos', {
      nombre: 'Nuevo producto',
      precio: 100,
    });
    console.log('Creado:', res.data);
  };

  return <ul>{productos.map(p => <li key={p.id}>{p.nombre}</li>)}</ul>;
}
```

fetch (nativo)	Axios (librería)
Hay que llamar <code>.json()</code> manualmente en cada respuesta	Parsea JSON automáticamente — accedés a <code>res.data</code>
Los errores 4xx/5xx NO se capturan en <code>.catch()</code>	Lanza errores para 4xx y 5xx automáticamente
Sin configuración global de baseURL o headers	Se puede configurar una instancia con baseURL y headers
Disponible en el navegador sin instalar nada	Requiere: <code>npm install axios</code>

💡 TIP

Para proyectos pequeños, `fetch` nativo es suficiente. Para apps medianas/grandes con muchas llamadas a una misma API, Axios con una instancia configurada ahorra mucho código repetitivo.



La mejor forma de aprender React es construyendo proyectos.

Tomá cada sección, escribí el código y rompé cosas — así se aprende.